Autonomous Surface Surveillance Boat

Group Members

MUHAMMAD UMER

Table of Contents

1. Abstract

We designed an Autonomous USV using ROS and Gazebo for maritime surveillance. The boat uses Lidar, GPS, and a Camera to detect targets and log their real-world coordinates. By combining computer vision (OpenCV) with sensor fusion, the system successfully tracks objects, navigates autonomously, and visualizes its path in real-time.

2. Introduction

We created "Floating World," a simulation to safely test robot boats (USVs) for dangerous missions. Using ROS, the system controls a boat that automatically searches for and chases targets. This serves as a prototype to validate GPS and camera logic for real-world use without risking hardware.

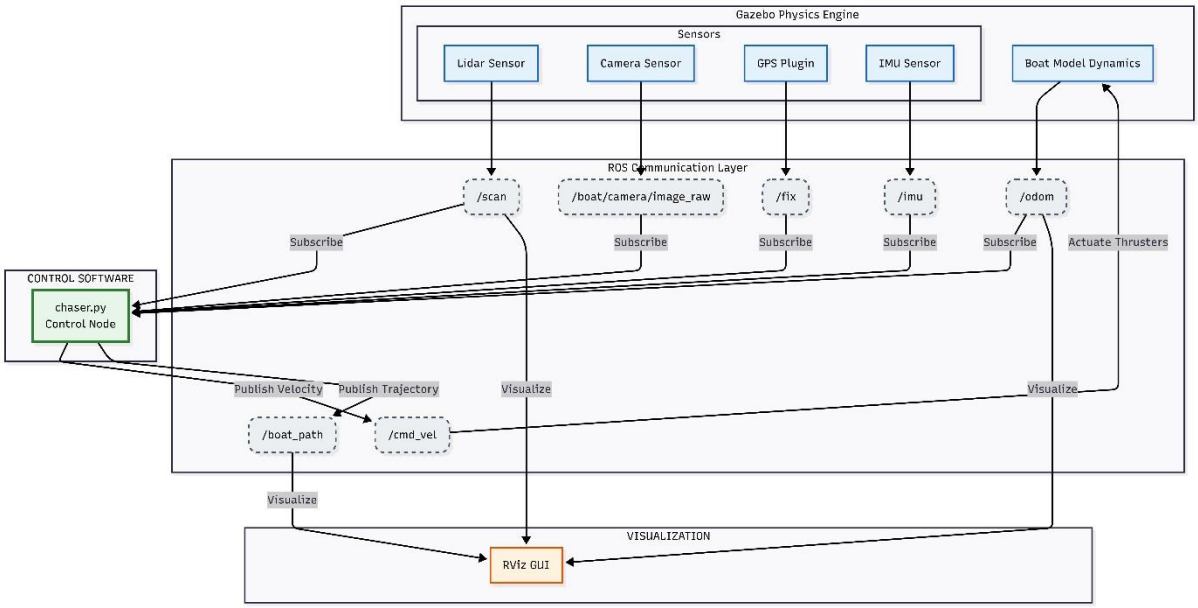
3. Literature Review

Research shows GPS lacks precision for close-range tasks, so we used **Sensor Fusion**, combining GPS tracking with Lidar distance measurement. For vision, we chose **HSV mode** over RGB, as studies prove it filters water reflections better. Finally, we followed ROS standards to ensure accurate path visualization in RViz.

4. Components and Tools Description

- **ROS Noetic:** The central operating system managing communication between the boat and the computer.
- **Gazebo Simulator:** A 3D physics engine used to simulate water, gravity, and the boat's physical movements.
- **Python (rospy):** Used to write the logic script (chaser.py) that controls the boat's behavior.
- **OpenCV:** A computer vision library used to detect the red target block from the camera feed.
- **RViz:** A visualization tool used to see what the Lidar sees and draw the boat's path.
- **Lidar Sensor:** Measures distance to obstacles and targets to prevent collisions.
- **GPS Plugin:** Simulates satellite data to provide real-time Latitude and Longitude.

5. Block Diagram/Flow Chart



6. Methodology

6.1. Proposed Model

The control logic follows a **Finite State Machine (FSM)** with four stages:

1. **Search:** The boat rotates to scan the area for visual targets.
2. **Chase:** Upon seeing a red object, the boat aligns itself and moves forward using a P-Controller.
3. **Capture:** When close (detected by Lidar), the boat stops and calculates the target's specific GPS coordinates.
4. **Return:** After finishing the mission, the boat calculates the vector back to home and navigates back.

6.2. Circuit / Simulation Description

The robot is a URDF model with a 0.15kg mass, utilizing planar_move and buoyancy plugins for stable movement on water.

The control node, chaser.py, steers the boat toward red targets using camera data. When close, it combines Lidar and heading data to calculate the target's specific GPS coordinates and visualizes the path history in RViz.

7. Results and Discussions

The simulation was successful. The boat remained stable on the water surface without sinking or flying, validating our physics settings.

- **Target Tracking:** The vision system accurately detected the red block. The log showed the robot transitioning from "SEARCHING" to "CHASING" instantly when the target appeared.

- **GPS Accuracy:** The system correctly initialized at the Karachi coordinates (24.8607, 67.0011). As the boat moved, the GPS updated in real-time, showing precise changes in the 5th decimal place (meters).
- **Visualization:** The path tracking worked perfectly. A green line appeared in RViz behind the boat, confirming that the Odometry and Path Publisher were synchronized.

8. Conclusion and Future Work

Conclusion:

This project successfully demonstrated an autonomous boat capable of visual navigation and GPS logging. We overcame challenges related to simulation physics (stability) and coordinate frames. The final result is a functional prototype that can patrol an area, identify objects of interest, and return to base automatically.

Future Work:

In the future, we plan to attach robotic arm The

robotic arm would:

• Attach to the boat model in Gazebo

• Use simple IK or predefined motions to grab the target

• Trigger only after the boat reaches the target

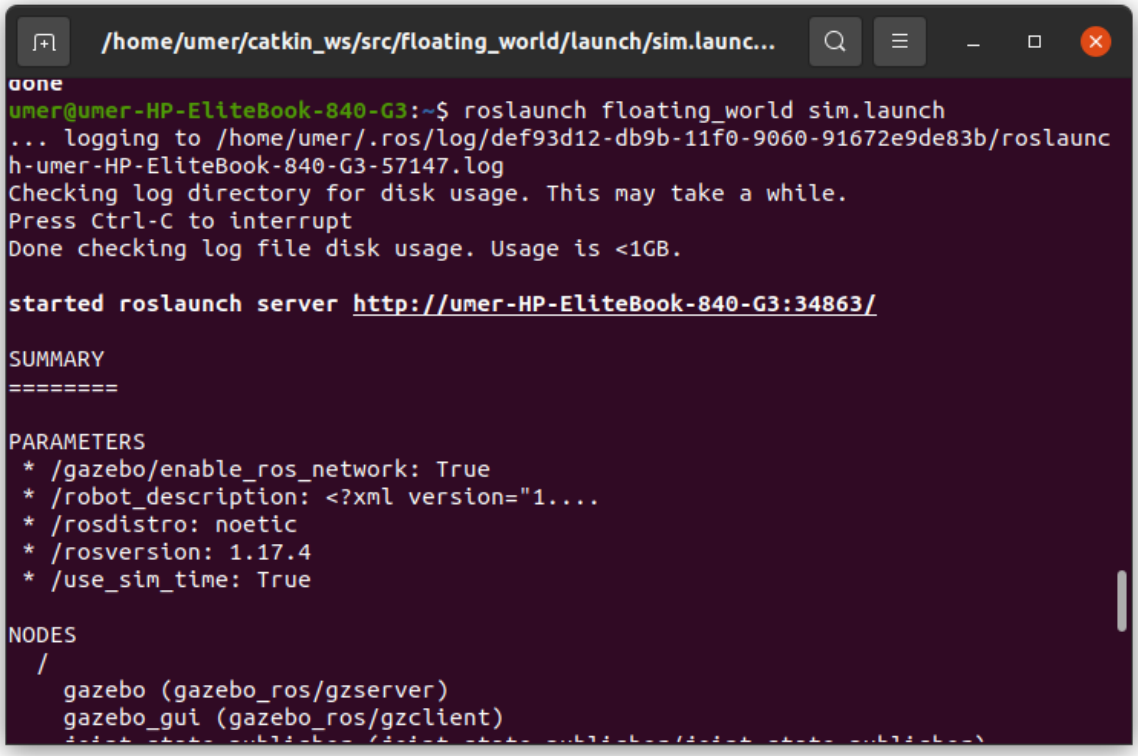
This extension can be included in the future if time and project stability allow

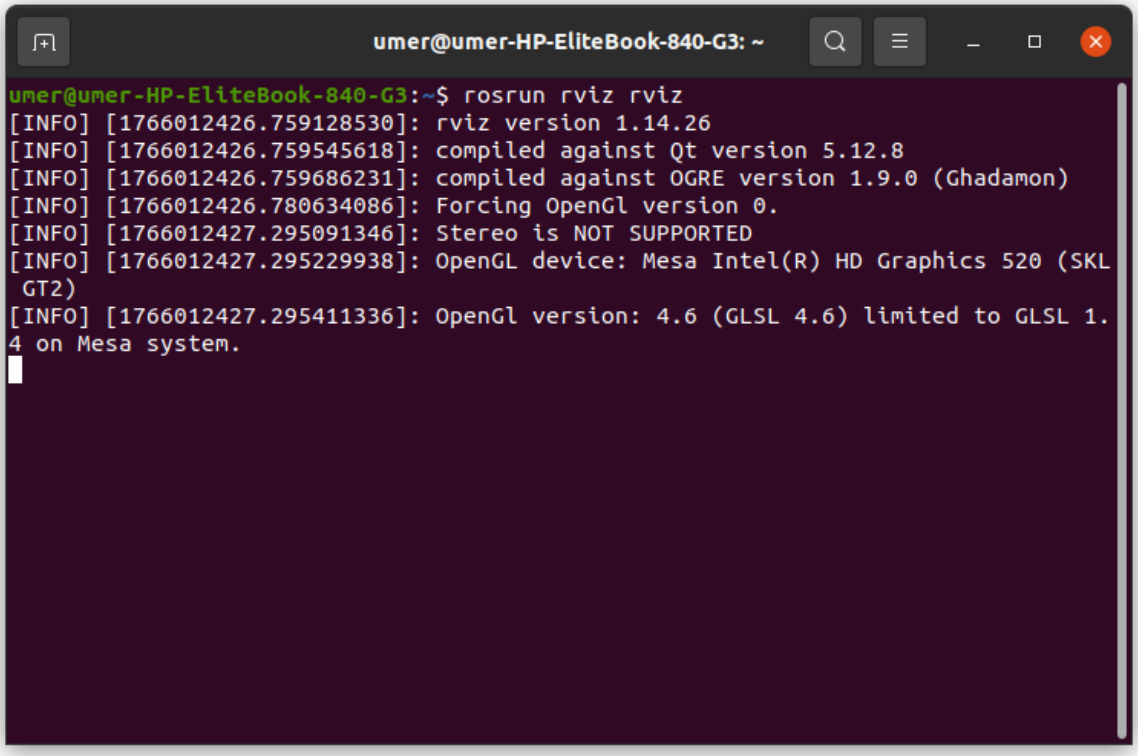
.**9. Project Summary**

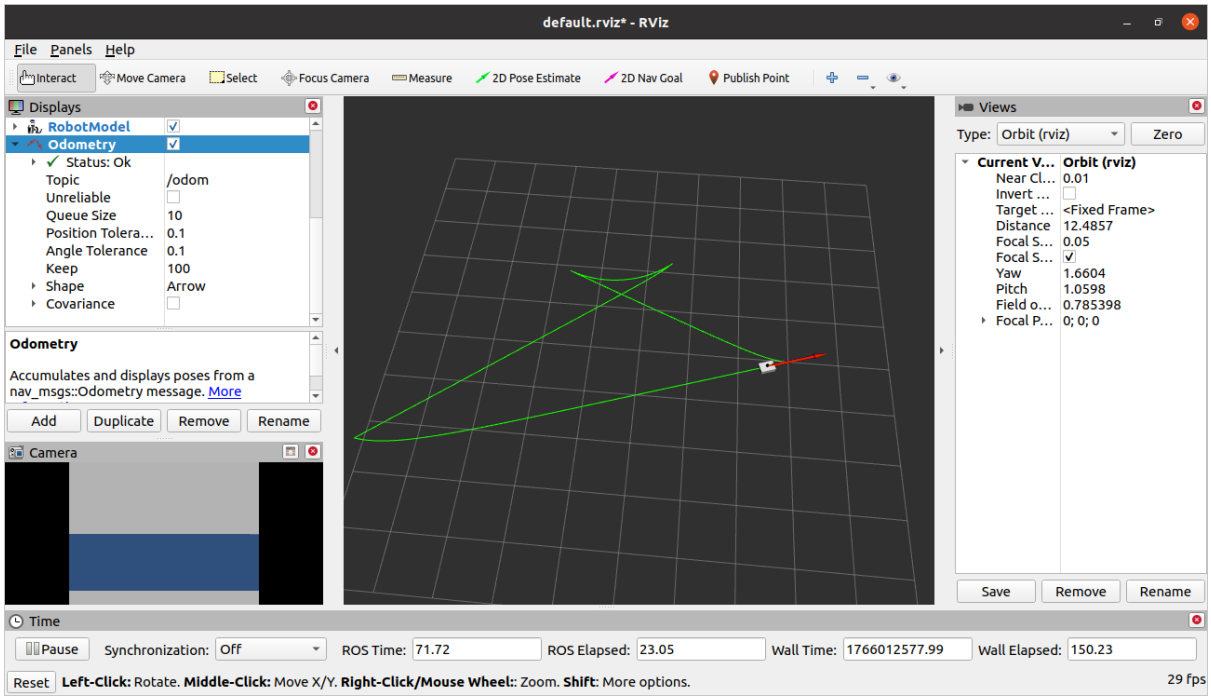
"Floating World" is a simulation project designed to create a smart boat. We built a custom ocean environment in Gazebo and a robot equipped with "eyes" (Camera) and "positioning" (GPS/Lidar). The boat uses Python code to autonomously find red objects.

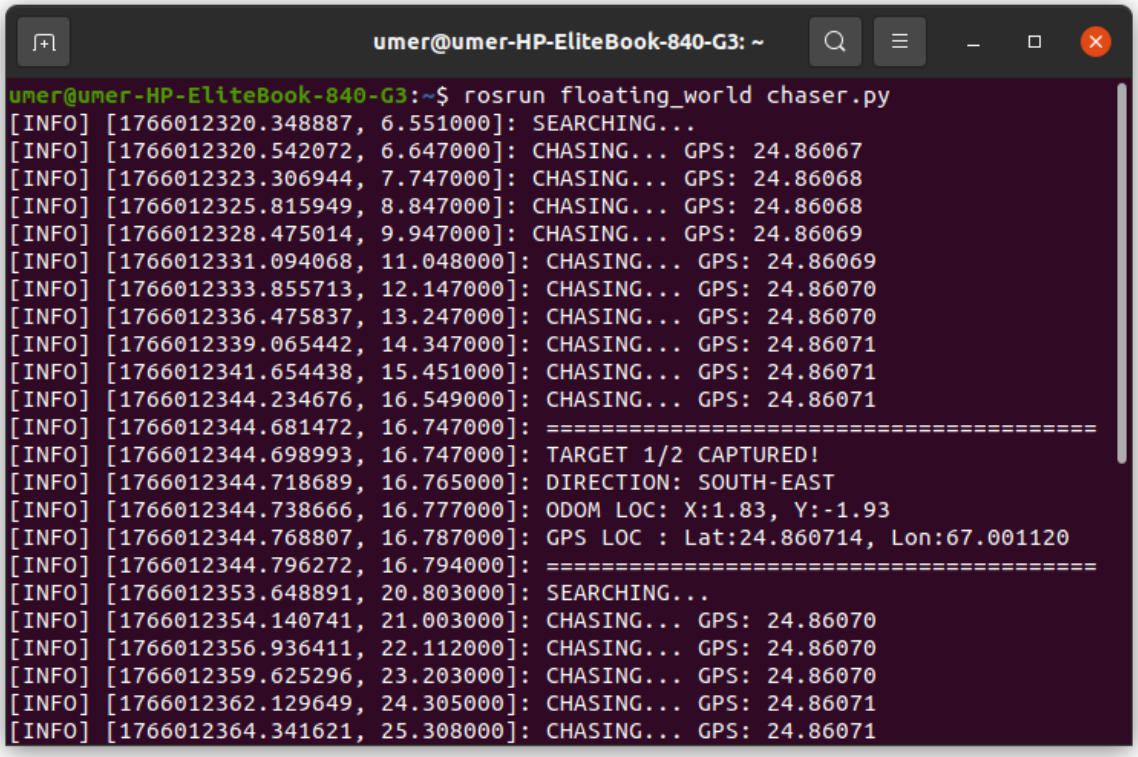
The unique achievement of this project is the integration of visual data with real-world coordinates; the boat doesn't just see an object, it calculates exactly where that object is on the map. We also added a live path tracker in RViz. Despite challenges with physics stability, the final system works reliably, proving that ROS is a powerful tool for marine robotics.

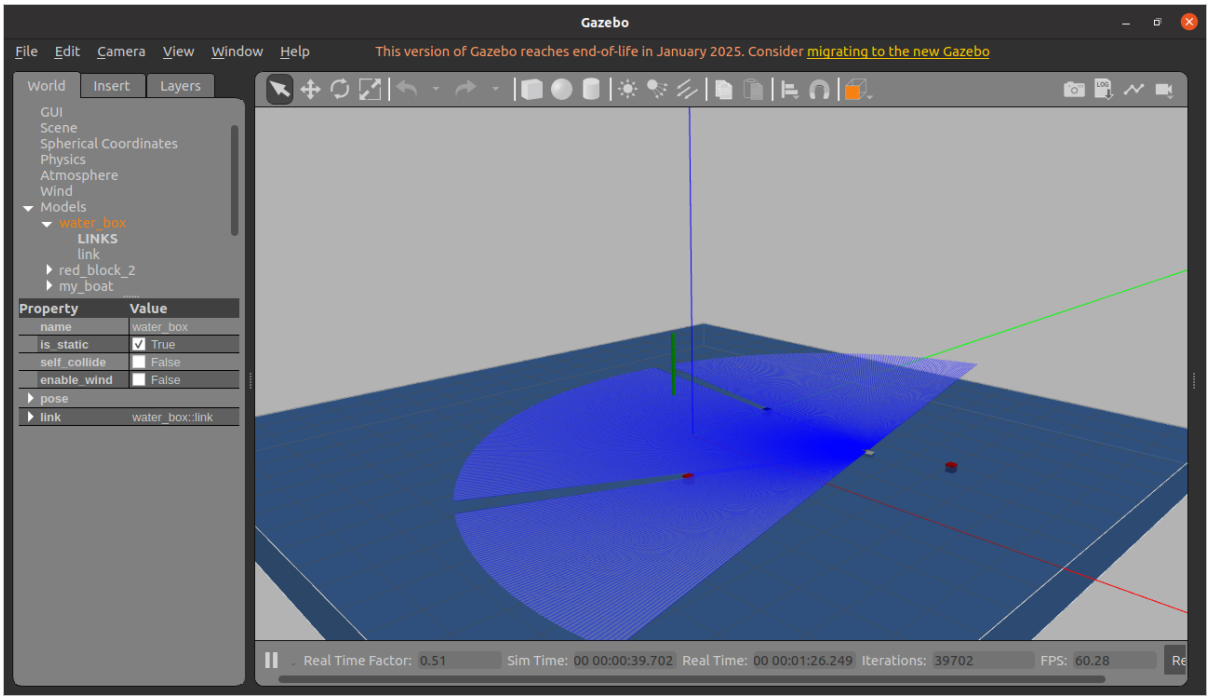
10. Project Pictures and working






default.rviz* - RViz
File Panels Help
Interact Move Camera Select Focus Camera Measure 2D Pose Estimate 2D Nav Goal Publish Point
Displays
RobotModel
Odometry
Status: Ok
Topic /odom
Unreliable
Queue Size 10
Position Tolera... 0.1
Angle Tolerance 0.1
Keep 100
Shape Arrow
Covariance
Odometry
Accumulates and displays poses from a nav_msgs::Odometry message. More
Add Duplicate Remove Rename
Camera
Views
Type: Orbit (rviz) Zero
Current V... Orbit (rviz)
Near Cl... 0.01
Invert ...
Target ... <Fixed Frame>
Distance 12.4857
Focal S... 0.05
Focal S...
Yaw 1.6604
Pitch 1.0598
Field o... 0.785398
Focal P... 0; 0; 0
Save Remove Rename
Time
Pause Synchronization: Off ROS Time: 71.72 ROS Elapsed: 23.05 Wall Time: 1766012577.99 Wall Elapsed: 150.23
Reset Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel:: Zoom. Shift: More options.
29 fps


umer@umer-HP-EliteBook-840-G3: ~
umer@umer-HP-EliteBook-840-G3:~$ rosrun floating_world chaser.py
[INFO] [1766012320.348887, 6.551000]: SEARCHING...
[INFO] [1766012320.542072, 6.647000]: CHASING... GPS: 24.86067
[INFO] [1766012323.306944, 7.747000]: CHASING... GPS: 24.86068
[INFO] [1766012325.815949, 8.847000]: CHASING... GPS: 24.86068
[INFO] [1766012328.475014, 9.947000]: CHASING... GPS: 24.86069
[INFO] [1766012331.094068, 11.048000]: CHASING... GPS: 24.86069
[INFO] [1766012333.855713, 12.147000]: CHASING... GPS: 24.86070
[INFO] [1766012336.475837, 13.247000]: CHASING... GPS: 24.86070
[INFO] [1766012339.065442, 14.347000]: CHASING... GPS: 24.86071
[INFO] [1766012341.654438, 15.451000]: CHASING... GPS: 24.86071
[INFO] [1766012344.234676, 16.549000]: CHASING... GPS: 24.86071
[INFO] [1766012344.681472, 16.747000]: ==
[INFO] [1766012344.698993, 16.747000]: TARGET 1/2 CAPTURED!
[INFO] [1766012344.718689, 16.765000]: DIRECTION: SOUTH-EAST
[INFO] [1766012344.738666, 16.777000]: ODOM LOC: X:1.83, Y:-1.93
[INFO] [1766012344.768807, 16.787000]: GPS LOC : Lat:24.860714, Lon:67.001120
[INFO] [1766012344.796272, 16.794000]: ==
[INFO] [1766012353.648891, 20.803000]: SEARCHING...
[INFO] [1766012354.140741, 21.003000]: CHASING... GPS: 24.86070
[INFO] [1766012356.936411, 22.112000]: CHASING... GPS: 24.86070
[INFO] [1766012359.625296, 23.203000]: CHASING... GPS: 24.86070
[INFO] [1766012362.129649, 24.305000]: CHASING... GPS: 24.86071
[INFO] [1766012364.341621, 25.308000]: CHASING... GPS: 24.86071



11. References

- ROS Wiki. (2022). *gps_common - ROS Wiki*. http://wiki.ros.org/gps_common
- Gazebo Simulator. (2022). *Tutorial: Hydrodynamics*. http://gazebosim.org/tutorials